

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



REPORT
PROGRAMMING INTERGRATION PROJECT

KNOWLEDGE RETRIEVAL AND QA FOR
COMPUTER SCIENCE COURSES UTILIZING
RAG

MAJOR: COMPUTER SCIENCE

THESIS COMMITTEE : COMPUTER SCIENCE

SUPERVISOR(s) : NGUYỄN HỨA PHÙNG, Ph.D.

—————o0o—————

STUDENT : NGUYỄN QUỐC HIỆU - 2352344
: NGUYỄN HỮU CẦU - 2352129

HO CHI MINH CITY, May 2024

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Problem statement	2
1.3	Goals	2
1.4	Scope	2
1.5	Thesis structure	2
2	Theoretical Background	3
2.1	Retrieval-Augmented Generation (RAG) pipeline	3
2.2	BERT-based Embeddings for Semantic Retrieval	4
2.2.1	Why BERT for Embeddings	4
2.2.2	Pooling and Normalization Strategies	5
2.2.3	Similarity Metrics	5
2.2.4	Embedding Dimension and Impact on Recall	6
2.2.5	Lightweight Alternatives: MiniLM	7
2.3	Vector Search	8
2.3.1	Vector-Based Retrieval Mechanism	9
2.3.2	Vector database	10
2.3.3	Role of Vector Search in Retrieval-Augmented Generation (RAG)	10
2.4	Reranking in Information Retrieval	11
2.4.1	Cross-Encoder vs Bi-Encoder Architectures	11
2.4.2	When Reranking is Needed	12

2.4.3	BGE Reranker	13
2.5	Large Language Models	13
3	Methods	16
3.1	Requirements Analysis	16
3.1.1	Functional Requirements	16
3.1.2	Non-Functional Requirements	17
3.1.3	Data Requirements	18
3.1.4	Constraints	18
3.2	System Design	19
3.2.1	System Architecture Overview	19
3.2.2	Component Design	19
3.2.2.1	Preprocessing Module	19
3.2.2.2	Embedding & Indexing Module	21
3.2.2.3	Query Pipeline Module	22
3.2.3	Data Flow	24
3.2.3.1	Indexing Flow	24
3.2.3.2	Query Flow	25
3.2.4	Technology Stack	25
3.2.5	Design Decisions	26
3.2.5.1	Embedding Model Selection	26
3.2.5.2	FAISS IndexFlatIP	26
3.2.5.3	Sliding Window Chunking	26
3.2.5.4	Two-Stage Retrieval (Retrieval + Reranking)	27
3.2.5.5	Modular Architecture	27
3.2.6	Deployment Architecture	27
4	Add-ons	28
	References	28

List of Figures

Figure 2.1	Bi-Encoder and Cross-Encoder architectures comparison [10]	11
Figure 2.2	BGE embedder and reranker in an RAG pipeline [10] .	13
Figure 2.3	The evolutionary tree of modern LLMs traces the development of language models in recent years [11] . .	14
Figure 3.1	Complete system architecture showing three main pipelines: Preprocessing (left), Embedding & Index (middle), and Query Pipeline → API/FE (right). The diagram illustrates data flow from document ingestion through indexing to query processing and response generation. .	20
Figure 3.2	Preprocessing pipeline showing document conversion, OCR processing, parsing, and chunking stages.	21
Figure 3.3	Embedding and indexing pipeline showing text-to-embedding conversion, L2 normalization, and storage in FAISS index with metadata.	22
Figure 3.4	Query pipeline showing client/Frontend interaction with API Backend, vector search in Index Store, reranking with BSS Reranker, and response generation.	23

List of Tables

Abstract

Chapter 1

Introduction

In Chapter 1, the rationale for choosing the topic, the overview, objectives and goals of the research's project are illustrated. The outline of the report is also presented.

1.1 Motivation

1.2 Problem statement

1.3 Goals

1.4 Scope

1.5 Thesis structure

Chapter 2

Theoretical Background

In Chapter 2, it presents about fundamental knowledge in this project.

2.1 Retrieval-Augmented Generation (RAG) pipeline

RAG decouples knowledge lookup from text generation by chaining two stages: (1) **retrieve** relevant passages from a knowledge source and (2) **generate** an answer conditioned on both the query and the retrieved context [1]. The typical flow is: user query $\rightarrow$ retriever selects top- k chunks $\rightarrow$ generator produces a grounded response.

- **Retriever role.** The retriever encodes the query and documents, scores similarity, and returns the most relevant contexts. Its objective is high recall under latency and cost constraints so that the generator receives useful evidence.
- **Generator role.** The generator (e.g., an LLM) consumes the query plus retrieved contexts and crafts the final answer. It should stay grounded in the provided evidence, cite or refer to the supporting snippets, and avoid hallucinations.

Feedback and iteration. In practice, systems may refine results via iterative retrieval (e.g., re-querying based on the first draft), reranking retrieved pas-

sages, or answer verification to detect low-confidence outputs and trigger a new retrieve–then–read round. This feedback loop helps improve factuality and coverage, especially for multi-hop or under-specified questions.

2.2 BERT-based Embeddings for Semantic Retrieval

BERT (Bidirectional Encoder Representations from Transformers) [2] has become a foundational model for generating dense embeddings in information retrieval systems. Unlike traditional word-level embeddings, BERT produces contextualized representations that capture bidirectional context, making it particularly suitable for semantic similarity tasks.

2.2.1 Why BERT for Embeddings

BERT is chosen for embedding generation due to several key advantages:

- **Bidirectional context:** Unlike unidirectional language models, BERT processes text in both directions simultaneously, enabling richer semantic understanding by considering both preceding and following context for each token.
- **Pre-trained knowledge:** BERT is pre-trained on large corpora using masked language modeling and next sentence prediction objectives, encoding substantial linguistic and world knowledge that transfers well to downstream tasks.
- **Task adaptability:** Through fine-tuning or feature extraction, BERT embeddings can be adapted for various retrieval tasks without requiring task-specific architectural changes.

However, vanilla BERT is not optimized for producing sentence-level embeddings suitable for similarity search. Sentence-BERT (SBERT) [3] addresses

this limitation by fine-tuning BERT with siamese and triplet network structures on sentence-pair datasets, enabling efficient semantic similarity computation.

2.2.2 Pooling and Normalization Strategies

Converting BERT's token-level outputs to fixed-size sentence embeddings requires pooling strategies:

- **Mean pooling:** Computes the average of all token embeddings (excluding special tokens). This approach captures information from all tokens and is empirically shown to outperform CLS token pooling for semantic similarity tasks [3].
- **CLS token pooling:** Uses the [CLS] token's embedding as the sentence representation. While designed for classification tasks, it often underperforms mean pooling for retrieval due to limited semantic information.
- **Max pooling:** Takes the maximum value across tokens for each dimension. Less commonly used for semantic tasks as it may emphasize outlier features.

Normalization is crucial for embedding quality. L2 normalization (unit vector scaling) is commonly applied to embeddings before similarity computation. This normalization:

- Reduces the impact of embedding magnitude on similarity scores
- Improves training stability and convergence
- Makes cosine similarity equivalent to dot product for normalized vectors

2.2.3 Similarity Metrics

The choice of similarity metric directly impacts retrieval performance:

- **Cosine similarity:** Measures the cosine of the angle between two vectors: $\cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|}$. For L2-normalized embeddings, this reduces to dot product. Cosine similarity is scale-invariant and widely used for semantic retrieval.
- **Inner product (dot product):** Computes $\mathbf{u} \cdot \mathbf{v} = \sum_i u_i v_i$. When embeddings are normalized, inner product equals cosine similarity. Inner product is computationally efficient and directly optimizable in vector search libraries like FAISS.
- **Euclidean distance:** Measures L_2 distance: $\|\mathbf{u} - \mathbf{v}\|_2$. Less common for semantic similarity as it is sensitive to vector magnitude.

For normalized embeddings, cosine similarity and inner product are equivalent and both are commonly used in production systems.

2.2.4 Embedding Dimension and Impact on Recall

The embedding dimension (typically 768 for BERT-base, 1024 for BERT-large) affects both retrieval quality and computational efficiency:

- **Higher dimensions** can capture more nuanced semantic information, potentially improving recall for complex queries. However, they increase storage requirements and search latency.
- **Lower dimensions** (achieved via dimensionality reduction) reduce computational cost but may lose semantic expressiveness, potentially decreasing recall.
- **Recall trade-offs:** In approximate nearest neighbor search (e.g., FAISS), higher-dimensional embeddings require more memory and may need larger index parameters (e.g., more centroids in IVF) to maintain recall. The choice of dimension balances between semantic expressiveness and search efficiency.

Empirical studies show that BERT-base embeddings (768 dimensions) with mean pooling and L2 normalization provide a good balance between quality and efficiency for most retrieval tasks, achieving strong recall while remaining computationally tractable for large-scale vector databases.

2.2.5 Lightweight Alternatives: MiniLM

While BERT-base and BERT-large provide high-quality embeddings, their computational requirements (110M and 340M parameters respectively) can be prohibitive for resource-constrained deployments. MiniLM [4] addresses this limitation through knowledge distillation, compressing larger transformer models into smaller, faster variants while preserving semantic quality.

MiniLM architecture: MiniLM uses deep self-attention distillation to transfer knowledge from teacher models (e.g., BERT-base) to student models with fewer layers and hidden dimensions. The distillation process focuses on matching attention distributions and hidden states, enabling the student to capture essential semantic patterns.

all-MiniLM-L6-v2: The all-MiniLM-L6-v2 model, part of the Sentence-Transformers library, is a distilled version optimized for sentence embeddings. Key characteristics:

- **Compact size:** 22.7M parameters (compared to 110M for BERT-base), enabling faster inference and lower memory footprint.
- **Lower dimensionality:** 384-dimensional embeddings (vs. 768 for BERT-base), reducing storage requirements and search latency in vector databases.
- **Performance:** Despite being smaller, MiniLM maintains competitive semantic quality on benchmark tasks, often achieving 95–98% of BERT-base performance with 2–3x faster inference.

Trade-offs and selection criteria: The choice between BERT-base and MiniLM involves balancing:

- **Quality vs. efficiency:** BERT-base offers marginally better semantic understanding for complex queries, while MiniLM provides faster batch processing and lower resource consumption.
- **Recall impact:** For most retrieval tasks, the recall difference between 384-dimensional MiniLM and 768-dimensional BERT-base embeddings is minimal (typically <2%), making MiniLM suitable for production systems prioritizing latency and cost.
- **Scalability:** Lower-dimensional embeddings enable more efficient FAISS indexing, allowing larger candidate sets to be searched within the same memory budget.

In practice, MiniLM is often preferred for production RAG systems where inference speed and resource efficiency are critical, while maintaining acceptable retrieval quality. The 384-dimensional embeddings provide sufficient semantic expressiveness for most domain-specific retrieval tasks while enabling real-time query processing.

2.3 Vector Search

Vector search [5] constitutes a fundamental paradigm for semantic information retrieval, in which documents and queries are represented as points in a continuous vector space and relevance is determined by geometric proximity rather than lexical overlap. Unlike traditional keyword-based retrieval methods, vector search enables the retrieval of semantically related content even when there is little or no surface-level term matching, making it particularly suitable for modern language model-based systems.

From a conceptual perspective, vector search should not be viewed merely as a nearest neighbor algorithm, but rather as a retrieval abstraction that integrates representation learning, similarity metrics, indexing strategies, and system-level

trade-offs. This abstraction forms the backbone of dense retrieval systems and plays a critical role in Retrieval-Augmented Generation (RAG) pipelines [1].

2.3.1 Vector-Based Retrieval Mechanism

In vector search, each document or text segment is mapped to a dense vector

$$\mathbf{v}_i \in \mathbb{R}^d$$

using an embedding model [3], while a user query is encoded into a query vector

$$\mathbf{q} \in \mathbb{R}^d$$

within the same vector space.

The objective of the retrieval process is to identify the set of document vectors that are most similar to the query vector. Formally, this can be expressed as:

$$\text{top-k}(\mathbf{q}) = \arg \max_{\mathbf{v}_i} s(\mathbf{q}, \mathbf{v}_i),$$

where $s()$ denotes a similarity function.

Common similarity measures include cosine similarity, inner product, and Euclidean distance. In practice, cosine similarity is widely adopted for semantic retrieval tasks due to its robustness to vector magnitude. When vectors are L2-normalized, cosine similarity becomes equivalent to the inner product:

$$\cos(\mathbf{q}, \mathbf{v}) = \frac{\mathbf{q} \cdot \mathbf{v}}{\|\mathbf{q}\| \|\mathbf{v}\|}.$$

Because embeddings generated by modern language models often reside in high-dimensional spaces, exhaustive linear search can become computationally expensive. As a result, vector search systems employ various indexing strategies and search algorithms, including exact search and approximate nearest neighbor methods, to balance retrieval accuracy and efficiency.

2.3.2 Vector database

With the growing adoption of vector-based retrieval, a variety of vector databases have been developed to support large-scale storage and search over embedding vectors. These systems typically integrate vector search capabilities with persistent storage, metadata management, conditional filtering, and distributed deployment, targeting production environments with dynamic and large-scale data. Representative vector database systems include *Qdrant* [6], which emphasizes approximate nearest neighbor search combined with payload-based filtering, *Milvus* [7], which provides a distributed architecture for large-scale vector data management, and *Weaviate* [8], which integrates vector search with schema-aware data modeling and API-based access. These systems illustrate a service-oriented approach in which vector search is tightly coupled with database-level functionalities.

In parallel, vector search libraries focus primarily on efficient similarity computation and indexing, without providing full database functionalities. Rather than serving as standalone data management systems, these libraries act as core computational components that can be embedded into broader retrieval pipelines.

FAISS is a representative example of vector search libraries, designed specifically for efficient similarity search over dense vectors in high-dimensional spaces. In the context of RAG systems, FAISS is often referenced as a canonical implementation of library-based vector search, while other vector database systems exemplify service-oriented approaches tailored for large-scale production use .

2.3.3 Role of Vector Search in Retrieval-Augmented Generation (RAG)

Within a Retrieval-Augmented Generation framework [1], vector search typically functions as the first-stage retriever. Its primary role is to identify a set of candidate text segments that exhibit high semantic relevance to a given query.

The effectiveness of this retrieval stage has a direct impact on subsequent components, such as reranking mechanisms and language model–based answer generation. A vector search system with high recall helps ensure that relevant contextual information is made available to downstream models, thereby supporting more accurate and contextually grounded responses.

For this reason, vector search is widely regarded as a foundational component in RAG architectures, serving as the critical link between semantic representation learning and generative language modeling.

2.4 Reranking in Information Retrieval

Reranking is a two-stage retrieval strategy that refines initial retrieval results by applying a more sophisticated scoring model to a small candidate set [9]. In the first stage, a fast retriever (typically a bi-encoder) retrieves a large set of candidates (e.g., top- k with $k = 100$). In the second stage, a reranker (typically a cross-encoder) re-scores these candidates to produce the final ranked list.

2.4.1 Cross-Encoder vs Bi-Encoder Architectures

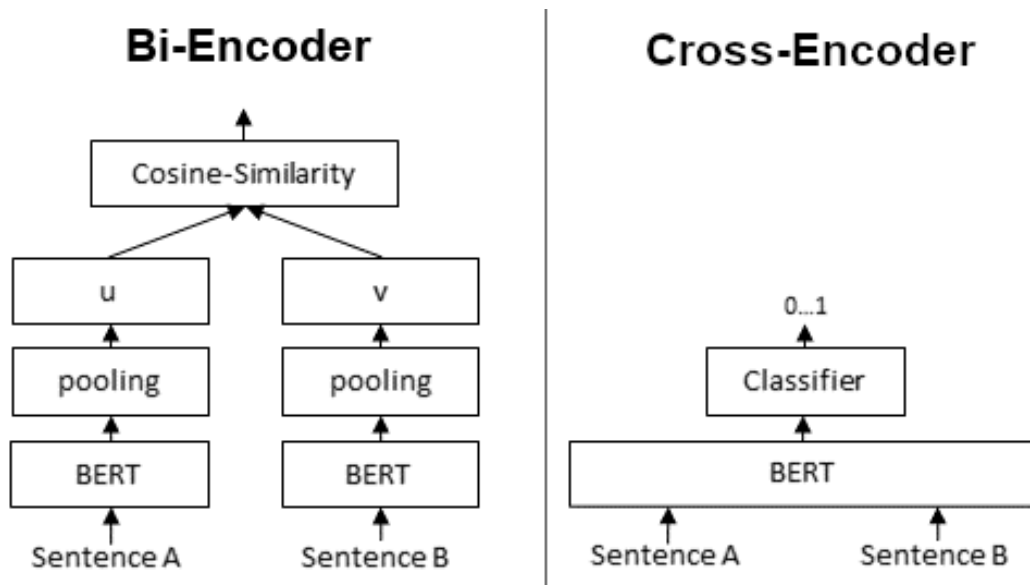


Figure 2.1: Bi-Encoder and Cross-Encoder architectures comparison [10]

The choice between cross-encoder and bi-encoder architectures represents a fundamental trade-off between accuracy and efficiency:

- **Bi-encoder** (used in initial retrieval): Query and document are encoded separately into dense vectors. Similarity is computed via dot product or cosine similarity. This enables efficient approximate nearest neighbor search (e.g., using FAISS, Qdrant) with sub-linear time complexity, making it suitable for large-scale retrieval. However, bi-encoders cannot model fine-grained query–document interactions.
- **Cross-encoder** (used in reranking): Query and document are concatenated and fed into a single transformer model, enabling joint attention over both inputs. This allows the model to capture complex semantic interactions and fine-grained relevance signals, typically achieving higher accuracy than bi-encoders. However, cross-encoders require computing scores for each query–document pair independently, making them computationally expensive for large candidate sets.

The two-stage approach leverages the strengths of both: bi-encoders provide fast candidate retrieval, while cross-encoders provide accurate final ranking on a small subset.

2.4.2 When Reranking is Needed

Reranking becomes particularly valuable when:

- The initial retriever has high recall but lower precision, requiring refinement to identify the most relevant passages.
- The downstream task (e.g., LLM generation) benefits from higher-quality context, where even small improvements in top- k relevance can significantly impact answer quality.

- Computational budget allows for the additional latency cost of reranking a small candidate set (typically 10–50 items).

2.4.3 BGE Reranker

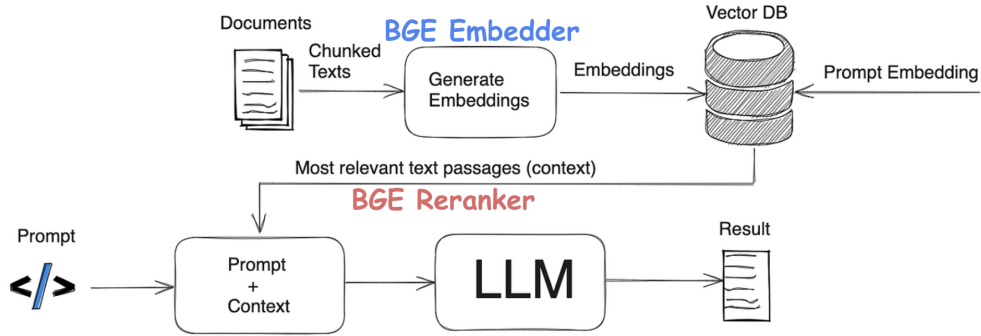


Figure 2.2: BGE embedder and reranker in an RAG pipeline [10]

The BGE (BAAI General Embedding) reranker is a cross-encoder model developed by the Beijing Academy of Artificial Intelligence (BAAI) [10]. Built on XLM-RoBERTa architecture, it is specifically designed for reranking tasks in both English and Chinese. The model employs a cross-encoder architecture where query and passage are concatenated and processed jointly, enabling fine-grained relevance scoring. BGE reranker models (base and large variants) have demonstrated state-of-the-art performance on various retrieval benchmarks, making them suitable for production RAG systems where accuracy is prioritized over minimal latency.

2.5 Large Language Models

Large Language Models that are pre-trained on large-scale corpus have demonstrated significant potential in various natural language processing tasks. Most LLMs are based on the Transformer design, which consists of encoder and decoder modules that are empowered by a self-attention mechanism. Based on their architecture structure, LLMs can be classified into three categories:

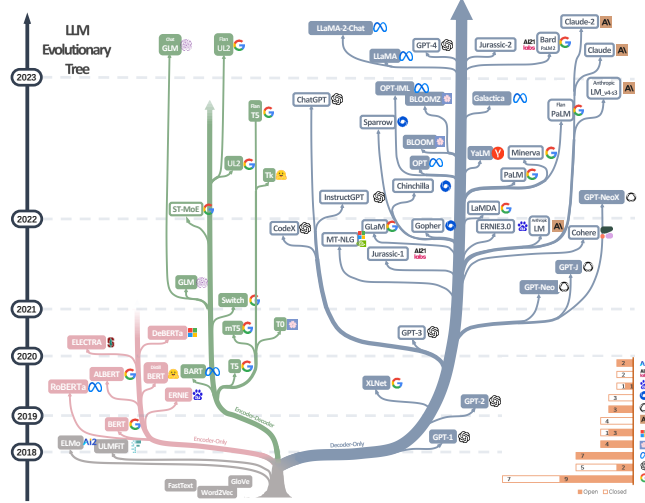


Figure 2.3: The evolutionary tree of modern LLMs traces the development of language models in recent years [11]
encoder-only LLMs, encoder-decoder LLMs, and decoder-only LLMs. Figure 2.3 summarizes several representative LLMs with different model architectures, model sizes, and open-source availabilities.

Encoder-only LLMs are a type of Large Language Models that use only the encoder to encode the sentence and understand the relationships between words. These models are typically pre-trained by predicting the mask words in an input sentence using large-scale corpus. To resolve downstream tasks, such as text classification and named entity recognition, an extra prediction head is added to encoder-only LLMs like BERT [2], RoBERTa [12], ALBERT [13], and ELECTRA [14]. These models are most effective for tasks that require understanding the entire sentence.

Encoder-decoder LLMs adopt both the encoder and decoder module. The encoder module is responsible for encoding the input sentence into a hidden space, and the decoder is used to generate the target output text. The training strategies in encoder-decoder LLMs can be more flexible. Encoder-decoder LLMs are able to directly resolve tasks that generate sentences based on some context,

such as summariaztion, translation, and question answering.

Decoder-only LLMs only adopt the decoder module to generate target output text. These models are trained to predict the next word in a sentence. They can perform downstream tasks with few examples or simple instructions without adding prediction heads or fine-tuning. [15]. Many state-of-the-art LLMs follow the decoder-only architecture. Recently Llama-2 [16], Alpaca ¹ and Vicuna ² are released as open-source decoder-only LLMs. These models are finetuned based on LLaMA [17] and achieve comparable performance with ChatGPT [18] and GPT-4

¹<https://crfm.stanford.edu/2023/03/13/alpaca.html>

²<https://lmsys.org/blog/2023-03-30-vicuna/>

Chapter 3

Methods

In Chapter 3, it presents the methodology for building the RAG system, including requirements analysis, system design, and detailed implementation of pre-processing, embedding, indexing, and LLM-powered query optimization.

3.1 Requirements Analysis

This section analyzes the functional and non-functional requirements for developing a RAG-based question-answering system tailored for Computer Science course materials at HCMUT.

3.1.1 Functional Requirements

The system is designed to collect course materials from multiple sources, including the HCMUT LMS and Coursewave, and will support various document formats such as PDF and PPTX. It will utilize OCR technology, specifically PaddleOCR, to extract text from scanned documents and parse syllabus documents to capture essential metadata. In order to facilitate semantic searches, the system will chunk text documents into overlapping segments, generate vector embeddings using a pre-trained model (sentence-transformers/all-MiniLM-L6-v2), and build FAISS vector indices for efficient similarity searching. Moreover, the system will maintain metadata mappings, linking vector IDs to original doc-

ument sources and other relevant information, allowing for both per-course and cross-course retrieval.

For query processing, the system will accept natural language questions from users and may employ a large language model (LLM) to rephrase queries for enhanced retrieval quality. It will retrieve the top-k document chunks based on semantic similarity and use a cross-encoder reranker to improve the relevance of these chunks, providing source citations for each response.

In terms of answer generation, the system will leverage a large language model to formulate answers based on the context obtained from the retrieved chunks. It will support multiple LLM providers, such as Google Gemini and Ollama, allowing for configurable models. The generated answers will include proper citation of sources and will handle cases where no relevant context is found by providing appropriate feedback.

The user interface will feature a web-based chat interface for user interactions, displaying conversation history and enabling follow-up questions. It will also show source citations for each answer and provide an API endpoint to allow programmatic access.

3.1.2 Non-Functional Requirements

The system is expected to efficiently retrieve relevant chunks within 500 milliseconds for queries against a corpus of over 10,000 chunks, generating answers within 5 seconds, encompassing retrieval, reranking, and answer formulation. The embedding model will produce embeddings of 384 dimensions, striking a balance between quality and efficiency, while also supporting batch processing for large document collections.

Regarding scalability, the system will facilitate indexing materials across more than 50 courses, support incremental updates of the vector index structure, and maintain a modular architecture to enable independent scaling of components.

To ensure reliability and availability, the system must gracefully handle API

errors and maintain data integrity during both indexing and retrieval operations. Health check endpoints will be provided for monitoring system performance and status.

Usability will be prioritized through an intuitive user interface that supports natural language queries without the need for specific syntax. Generated responses will be clear, concise, and adequately formatted. The codebase will follow modular design principles for easy maintenance and be containerized using Docker for consistent deployment across various environments.

3.1.3 Data Requirements

The input data will comprise course syllabi and lecture materials in PDF and PPTX formats, allowing for the extraction of chapter numbers, page indices, and hierarchical content organization. The output will include structured JSON files for parsed content, vector embeddings stored in FAISS indices with the associated metadata, and generated answers formatted with source citations for API responses.

To maintain data quality, the OCR process will focus on achieving sufficient accuracy for extracting readable text from scanned documents, while the chunking strategy will aim for semantic coherence within the chunks. Metadata must also be complete and accurate to ensure proper source attribution.

3.1.4 Constraints

The system will be designed to run efficiently on CPU without requiring GPU resources for inference. It will be focused specifically on Computer Science course materials at HCMUT, with no need for generalization to other domains. The knowledge base will be kept static, and real-time updates from external sources will not be supported. The availability of LLM APIs will rely on external services (like Gemini API) or local setups (such as Ollama).

3.2 System Design

This section presents the overall architecture and design of the RAG system for CSE course materials. The system is designed as a modular pipeline with distinct stages for data processing, indexing, and query answering.

3.2.1 System Architecture Overview

The system follows a three-stage architecture:

1. **Preprocessing Pipeline:** Processes raw documents, extracts text using OCR, parses structured information, and chunks text for indexing.
2. **Embedding & Index Pipeline:** Converts text chunks to vector embeddings, normalizes them, and stores them in FAISS indices with metadata.
3. **Query Pipeline:** Handles user queries through API/Frontend, retrieves relevant chunks, reranks results, and generates answers using LLM.

Figure 3.1 illustrates the complete system architecture with all three main pipelines and their interactions.

3.2.2 Component Design

3.2.2.1 Preprocessing Module

The preprocessing module handles document ingestion and text extraction. Figure 3.2 shows the detailed preprocessing pipeline.

The preprocessing module consists of:

- **Document Converter:** Converts PDF/DOCX/PPTX files to page images (220 DPI) for OCR processing.
- **OCR Engine:** Uses PaddleOCR 3.2 to extract text with bounding box information and confidence scores.

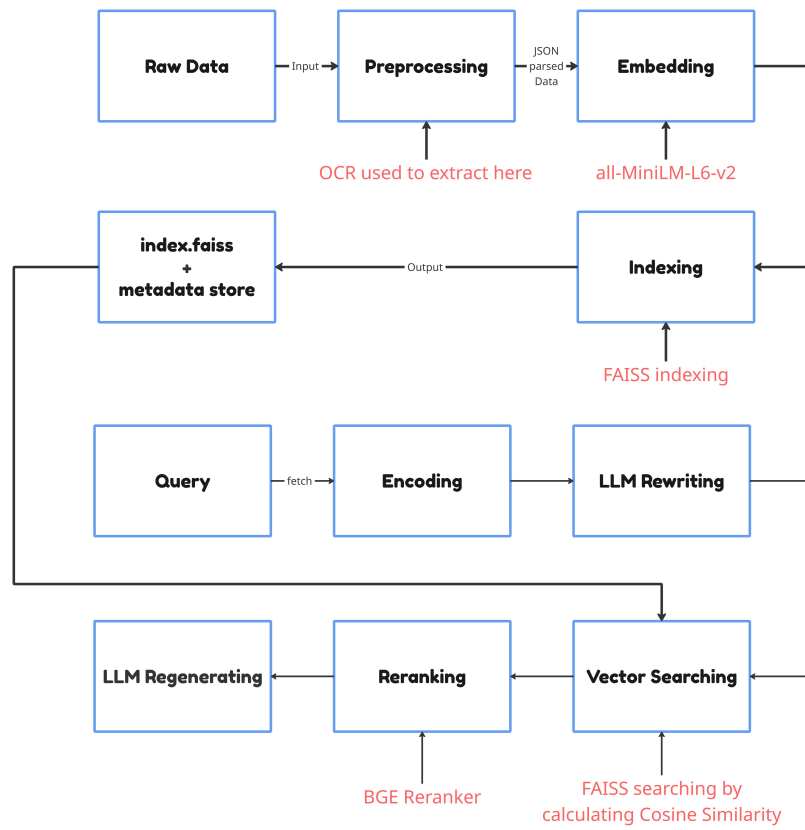


Figure 3.1: Complete system architecture showing three main pipelines: Pre-processing (left), Embedding & Index (middle), and Query Pipeline → API/FE (right). The diagram illustrates data flow from document ingestion through indexing to query processing and response generation.

- **Syllabus Parser:** Extracts structured information from syllabus documents including course metadata, grading policies, and learning objectives.
- **Material Parser:** Processes lecture slides to extract chapter information, page indices, and content structure.
- **Text Chunker:** Segments documents into overlapping chunks using a sliding window approach with configurable window size and overlap.

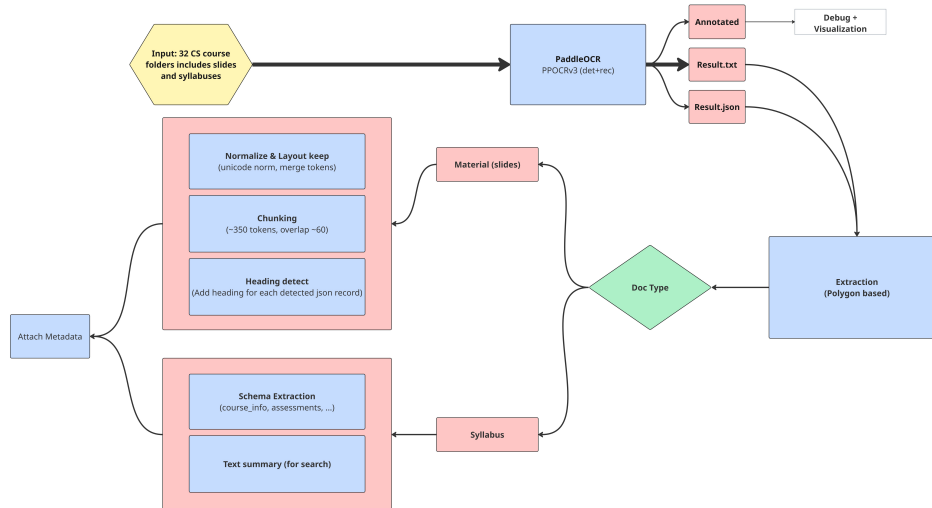


Figure 3.2: Preprocessing pipeline showing document conversion, OCR processing, parsing, and chunking stages.

3.2.2.2 Embedding & Indexing Module

The embedding and indexing modules work together to create searchable vector representations. Figure 3.3 illustrates the embedding generation, normalization, and indexing process.

The embedding module converts text into vector representations:

- **Embedding Model:** Uses sentence-transformers/all-MiniLM-L6-v2 to generate 384-dimensional embeddings.
- **Batch Processor:** Processes chunks in batches for efficient embedding generation.
- **Normalization:** L2-normalizes embeddings using the formula $\hat{x} = x / \|x\|_2$ to create unit vectors, enabling cosine similarity via inner product.

The indexing module builds and manages vector search indices:

- **FAISS Index:** Uses IndexFlatIP (Inner Product) for exact similarity search on normalized vectors.
- **Metadata Store:** Maintains JSON mappings from vector IDs to document metadata (course ID, page index, chapter, source file).



Figure 3.3: Embedding and indexing pipeline showing text-to-embedding conversion, L2 normalization, and storage in FAISS index with metadata.

- **Index Manager:** Handles index creation, updates, and persistence to disk.

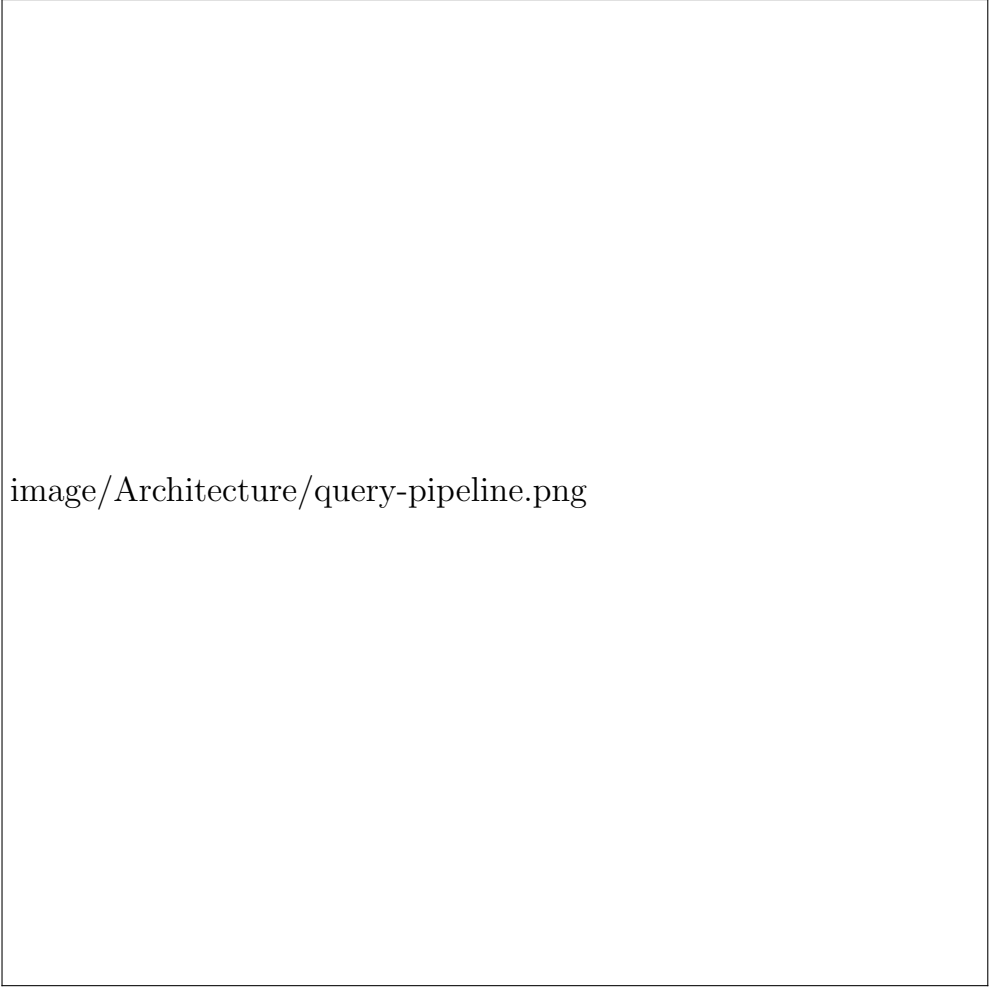
3.2.2.3 Query Pipeline Module

The query pipeline module handles the complete flow from user query to response generation. Figure 3.4 shows the detailed query processing architecture including API backend, reranking, and frontend interaction.

The query pipeline consists of several interconnected modules:

Retrieval Module handles query processing and document retrieval:

- **Query Rewriter** (optional): Uses LLM to rewrite user queries for improved retrieval quality.
- **Query Embedder:** Converts queries to embeddings using the same model



image/Architecture/query-pipeline.png

Figure 3.4: Query pipeline showing client/Frontend interaction with API Backend, vector search in Index Store, reranking with BSS Reranker, and response generation.

as document embeddings.

- **Vector Searcher:** Performs top- k similarity search using FAISS (search query) and fetches results by IDs.
- **Reranker:** Uses FlagEmbedding cross-encoder (BSS Reranker) to rerank retrieved chunks by relevance.

Generation Module produces answers from retrieved context:

- **LLM Client:** Abstracts interactions with different LLM providers (Gemini, Ollama).
- **Context Formatter:** Prepares retrieved chunks as context for the LLM prompt.

- **Response Generator:** Calls LLM with formatted prompt to generate answers.
- **Source Attacher:** Attaches source citations to generated answers.

API Module exposes system functionality via REST endpoints:

- **FastAPI Application:** Provides HTTP endpoints for query processing.
- **Health Check:** Monitors system status and component availability.
- **Course Listing:** Returns available courses in the knowledge base.
- **Query Endpoint:** Accepts questions and returns answers with sources.

Frontend Module provides user interface:

- **React Application:** Modern web interface built with React 19 and Vite.
- **Chat Interface:** Interactive conversation UI with message history.
- **Source Display:** Shows source citations for each answer.
- **API Integration:** Communicates with backend via REST API.

3.2.3 Data Flow

3.2.3.1 Indexing Flow

The indexing pipeline processes documents through the following stages:

1. **Document Ingestion:** Raw documents (PDF/DOCX/PPTX) are placed in `data/raw/<CourseName>/`.
2. **Conversion:** Documents are converted to page images stored in `data/converted/`.
3. **OCR Processing:** PaddleOCR extracts text and structure, producing JSON files in `data/data/`.

4. **Parsing:** Syllabus and material parsers extract structured information.
5. **Chunking:** Text is segmented into overlapping chunks with metadata.
6. **Embedding:** Chunks are converted to vector embeddings.
7. **Indexing:** Embeddings are stored in FAISS indices with metadata mappings in `data/indices/`.

3.2.3.2 Query Flow

The query pipeline processes user questions as follows:

1. **Query Input:** User submits a natural language question via UI or API.
2. **Query Rewriting** (optional): LLM rewrites the query for better retrieval.
3. **Query Embedding:** Query is converted to a vector embedding.
4. **Vector Search:** FAISS retrieves top- k similar chunks (default $k = 10$).
5. **Reranking:** Cross-encoder reranker scores and reranks retrieved chunks.
6. **Context Preparation:** Top reranked chunks are formatted as context.
7. **Answer Generation:** LLM generates answer using retrieved context.
8. **Response Formatting:** Answer and source citations are formatted and returned.

3.2.4 Technology Stack

The system uses the following technologies:

- **Processing:** Python 3.x, PaddleOCR 3.2, pdf2image, LibreOffice
- **Embedding:** sentence-transformers (all-MiniLM-L6-v2), FlagEmbedding (reranker)

- **Vector Search:** FAISS (Facebook AI Similarity Search)
- **Backend:** FastAPI, Pydantic, Uvicorn
- **LLM Providers:** Google Gemini API, Ollama (local)
- **Frontend:** React 19, Vite, TailwindCSS
- **Containerization:** Docker, Docker Compose

3.2.5 Design Decisions

3.2.5.1 Embedding Model Selection

We chose all-MiniLM-L6-v2 because:

- Compact 384-dimensional embeddings balance quality and efficiency.
- Fast inference suitable for CPU-only environments.
- Good semantic understanding for educational content.

3.2.5.2 FAISS IndexFlatIP

We use IndexFlatIP because:

- Provides exact (non-approximate) similarity search.
- Efficient for our corpus size (10,000+ chunks).
- Simple and reliable, no hyperparameter tuning required.
- Works well with L2-normalized embeddings (cosine = inner product).

3.2.5.3 Sliding Window Chunking

We use overlapping chunks because:

- Preserves context across sentence boundaries.
- Reduces information loss at chunk boundaries.
- Improves retrieval recall for queries spanning multiple sentences.

3.2.5.4 Two-Stage Retrieval (Retrieval + Reranking)

We combine FAISS retrieval with cross-encoder reranking because:

- FAISS provides fast candidate retrieval from large corpus.
- Cross-encoder reranker improves precision by considering query-document interactions.
- Better accuracy than single-stage retrieval alone.

3.2.5.5 Modular Architecture

We design the system as independent modules because:

- Enables parallel development and testing.
- Allows easy replacement of components (e.g., different embedding models).
- Facilitates maintenance and future extensions.
- Supports containerized deployment with separate services.

3.2.6 Deployment Architecture

The system is deployed using Docker Compose with the following services:

- **Backend Service:** FastAPI application running on port 8000.
- **Frontend Service:** React application served via Vite on port 5173.
- **Shared Volumes:** Data directory mounted for access to indices and processed documents.

Configuration is managed through environment variables, allowing easy adaptation to different deployment environments and LLM providers.

Chapter 4

Add-ons

Source code repository:  https://github.com/HatakekkSheeshh/CSE_course_RAG.git

References

- [1] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive nlp tasks”, in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding”, in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds., Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186.
- [3] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks”, in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.
- [4] W. Wang et al., “Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers”, *arXiv preprint arXiv:2002.10957*, 2020.
- [5] M. Douze et al., *The faiss library*, 2025. arXiv: [2401.08281](https://arxiv.org/abs/2401.08281) [cs.LG].
- [6] Qdrant Team, *Qdrant: Vector database and similarity search engine*, <https://qdrant.tech/>, Accessed: 2024, 2024.

- [7] J. Wang et al., “Milvus: A purpose-built vector data management system”, in *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*, ACM, 2021, pp. 2614–2627.
- [8] Weaviate Team, *Weaviate: Open-source vector database*, <https://weaviate.io/>, Accessed: 2024, 2024.
- [9] R. Nogueira and K. Cho, “Passage re-ranking with bert”, in *arXiv preprint arXiv:1901.04085*, 2019.
- [10] Beijing Academy of Artificial Intelligence, *Bge reranker: Baai general embedding reranker*, <https://github.com/FlagOpen/FlagEmbedding>, Accessed: 2024, 2024.
- [11] J. Yang et al., “Harnessing the power of llms in practice: A survey on chatgpt and beyond”, 2023.
- [12] Y. Liu et al., “Roberta: A robustly optimized bert pretraining approach”, 2019.
- [13] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, “Al-bert: A lite bert for self-supervised learning of language representations”, 2019.
- [14] K. Clark, M.-T. Luong, Q. V. Le, and C. D. Manning, “ELECTRA: Pre-training text encoders as discriminators rather than generators”, in *International Conference on Learning Representations*, 2020.
- [15] H. Liu et al., “Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning”, in *Advances in Neural Information Processing Systems*, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, Eds., 2022.
- [16] H. Touvron et al., “Llama 2: Open foundation and fine-tuned chat models”, 2023.
- [17] H. Touvron et al., “Llama: Open and efficient foundation language models”, 2023.

- [18] L. Ouyang et al., “Training language models to follow instructions with human feedback”, in *Advances in Neural Information Processing Systems*, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, Eds., 2022.